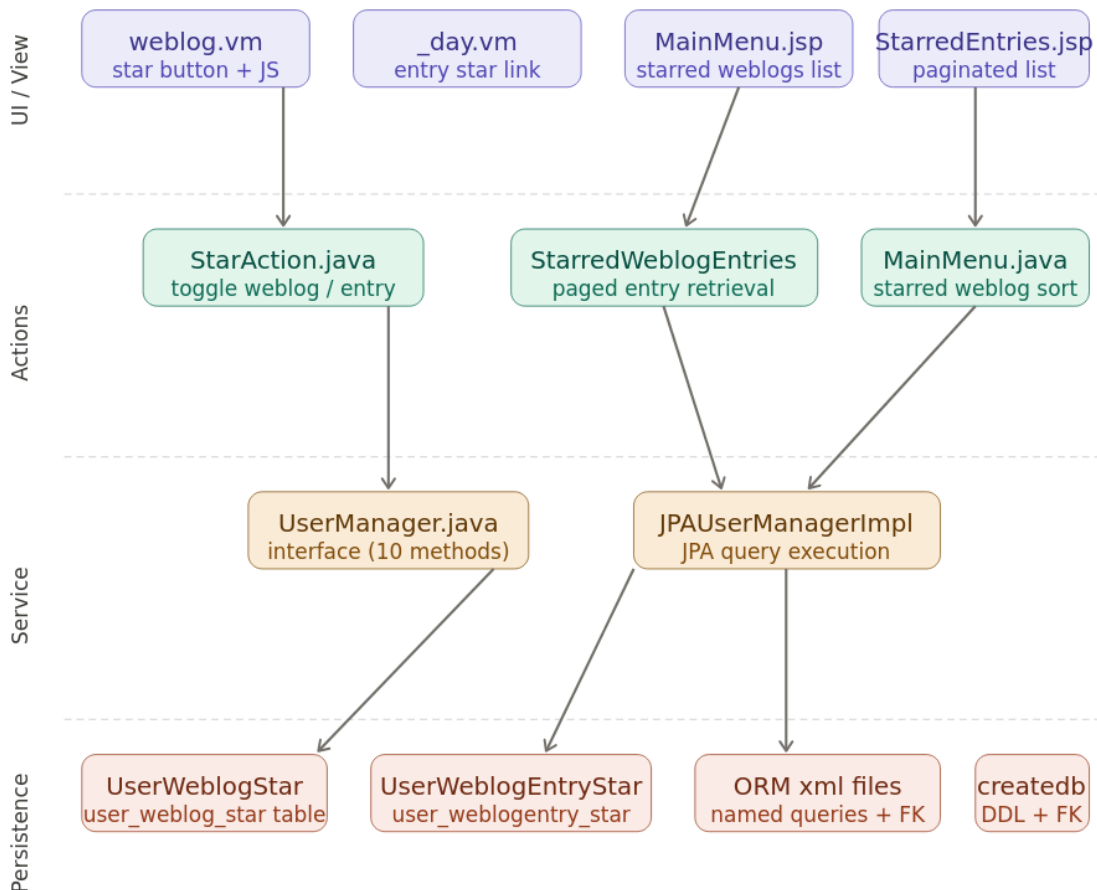


Project 2

Feature 1A - Stars



Feature Overview

The 1A Stars feature adds a social bookmarking/favouriting mechanism to Apache Roller. Users can star individual weblogs (blogs) and individual weblog entries (posts). Stars are per-user, so each user maintains their own set of starred items. The feature exposes this through two surfaces: a "Starred Weblogs" section on the main menu dashboard (sorted by most-recently-updated blog first), and a dedicated paginated "Starred Blogposts" page accessible from the navigation bar. A lightweight AJAX interaction means the star count updates in-place without a full page reload; unauthenticated clicks redirect to login.

Code Changes

Files Created (7 new files)

UserWeblogStar.java: A new JPA entity (POJO) representing the many-to-many join between a User and a Weblog. Stores a UUID primary key plus foreign key references to both. Implements equals/hashCode via Apache Commons builders on the user+weblog pair.

UserWeblogEntryStar.java: Identical in structure to the above but joins User to WeblogEntry instead, mapping to the user_weblogentry_star table.

UserWeblogStar.orm.xml: JPA ORM descriptor for the weblog star entity. Declares the table name, three named JPQL queries (getByUser, getByUserAndWeblog, getCountByWeblog), and the two many-to-one column mappings.

UserWeblogEntryStar.orm.xml: Same pattern for the entry star entity, with its own three named queries.

StarAction.java: A Struts2 action class with two methods: weblog() and entry(). Each method checks authentication, looks up the target object by ID, toggles the star (star if not starred, unstar if already starred), flushes the persistence context, invalidates the cache, and returns the new star count as plain text for AJAX callers. If the request is not XHR, it redirects back to the referer.

StarredWeblogEntries.java: A Struts2 action that fetches a paginated list of the authenticated user's starred blog entries (10 per page), setting a hasMore flag to drive the pagination UI.

StarredWeblogEntries.jsp: The JSP view for the above action. Iterates starred entries, showing title, blog name, publication date, and a "Remove Star" link. Includes previous/next pager controls.

Files Modified (16 files)

UserManager.java (interface): Added 10 new method signatures covering star, unstar, isStarred, getStarred, and getStarCount for both weblogs and weblog entries.

JPAUserManagerImpl.java: Implemented all 10 new interface methods using the named JPQL queries from the ORM descriptors. Uses getSingleResult with NoResultException catch for existence checks; uses getResultList for counts to avoid NoResultException on aggregates.

persistence.xml: Registered both new ORM xml mapping files so JPA picks them up at startup.

createdb.vm: Added DDL for user_weblog_star and user_weblogentry_star tables (id, userid, weblogid/weblogentryid) plus four foreign key constraints referencing roller_user and weblog/weblogentry.

struts.xml: Registered the star action (with entry and weblog allowed methods) and the starredWeblogEntries action with its tile result. Added a global login redirect result.

files.xml: Added a .StarredWeblogEntries tile definition extending the main menu page layout.

WeblogWrapper.java: Added getStarCount() delegating to UserManager.getWeblogStarCount, callable from Velocity templates via \$model.weblog.starCount.

WeblogEntryWrapper.java: Same addition for getWeblogEntryStarCount, callable as \$entry.starCount.

MainMenu.java: Added getStarredWeblogs() (fetches starred weblogs, sorts descending by most recent entry pub date using a Comparator), getLatestEntry() (helper), and getHasStarredBlogposts() (boolean gate for the UI link).

MainMenu.jsp: Added a "Starred Weblogs" section showing each starred blog with its latest post date; added a "Starred Blogposts" link gated on hasStarredBlogposts.

bannerStatus.jsp: Added a "Starred Posts" nav link for authenticated users, pointing to the starredWeblogEntries action.

weblog.vm, fauxcoly/weblog.vm, frontpage/weblog.vm, gaurav/weblog.vm: Each gets the starItem() JavaScript AJAX function and a "Star this Blog" anchor with a live count span.

basic/_day.vm, fauxcoly/day.vm, gaurav/day.vm, frontpage/_entry.vm: Each gets a "Star this Entry" anchor with a live count span wired to starItem().

docker-compose.yml: Added a rollernet Docker network and a health check on the PostgreSQL container so the roller waits for it to be ready before starting.

Quality Attributes

Modularity: The feature is cleanly separated into its own layers: two new POJOs, two ORM descriptors, two new action classes, and two new JSP views. None of which modify the core Roller business logic. The starring concern sits alongside existing functionality without entangling it.

Maintainability: The toggle logic, count retrieval, and authentication check are all centralized in StarAction.java rather than scattered across JSPs or templates. The JPA named queries are declared in ORM XML files rather than hardcoded strings in Java, making query changes locatable and isolated.

Extensibility: The UserManager interface acts as a clean extension point. Adding new star targets in the future follows the exact same pattern: add methods to the interface, implement

them in JPAUserManagerImpl, create a new POJO and ORM descriptor. The architecture does not need structural changes to accommodate new star types.

Performance: The StarredWeblogEntries action fetches length + 1 records in a single query to determine whether more pages exist, avoiding a separate COUNT query. The CacheManager.invalidate call after each toggle ensures subsequent page loads reflect updated counts without serving stale cached data.

Feature 1B - Trending Blogs

Feature Overview

The Trending Blogs feature displays the Top 5 most-starred blog pages and the Top 5 most-starred blogposts across the entire site. Rankings are based on the number of users who have starred each item using the existing star system.

A central requirement of this feature is efficiency: the data must not be fetched by iterating over individual articles. Instead, all aggregation and ranking is delegated to the database using GROUP BY JPQL queries, so Java receives a maximum of 5 result rows per query regardless of how many total stars exist in the system.

Code Changes

Files Created

TrendingBlogs.java - A new Struts2 action class in the core UI package. On request, it calls two methods on the existing UserManager service to retrieve the top-5 starred blog pages and blogposts, then makes both lists available to the view. It declares that neither a logged-in user nor a specific weblog is required, making the page publicly accessible.

TrendingBlogs.jsp - A new JSP view that renders two sections: one for trending blog pages and one for trending blogposts. Each item displays its title or name, star count, and a direct link. If no items have been starred yet, a friendly empty-state message is shown in place of each list.

Files Modified

UserWeblogEntryStar.orm.xml - A new named JPQL query (getTopByStarCount) was added. It groups all blogpost-star records by blogpost and counts the stars per blogpost, returning the results ordered by count descending. This is the query that powers the trending blog posts ranking.

UserWeblogStar.orm.xml - A new named JPQL query (getTopByStarCount) was added following the same structure as above, but grouping by blog page instead of blogpost. This powers the trending blog pages ranking.

UserManager.java - Two new method signatures were added to the interface: one for retrieving the top-N starred blogposts and one for the top-N starred blog pages, both accepting a limit parameter and returning a list of entity–count pairs.

JPAUserManagerImpl.java - Implementations of the two new interface methods were added. Each method invokes its corresponding named query and applies setMaxResults to cap the result at the database level before any data is returned to the application. A jakarta.persistence.Query import was also added.

struts.xml - A new action mapping was registered for the trendingBlogs action, pointing to the new TrendingBlogs class and mapping its success result to the new tile definition.

tiles.xml - A new tile definition .TrendingBlogs was added, extending the standard main-menu page layout and pointing to the new JSP for its content area.

MainMenu.jsp - A Trending button was added to the top navigation row of the main dashboard, alongside the existing “View Starred Blogposts” button, giving all users immediate access to the trending page.

Quality Attributes

Performance

The most significant quality improvement is performance efficiency. Rather than loading all star records and computing rankings in the application layer, the GROUP BY queries push aggregation, counting, and ordering entirely into the database engine. The database uses indexes on the foreign key columns to process the grouping efficiently. Java receives at most 5 rows per query, the cost in application memory and processing is constant regardless of how large the star tables grow over time.

Extensibility

The feature is designed to be easily extended. The top-N limit is passed as a parameter (limit) rather than hardcoded in the query, so changing the ranking from Top 5 to Top 10 requires a one-line change in the action class. Adding a new trending dimension such as trending-this-week or trending-by-category requires only a new named query in the ORM XML and a corresponding method in the interface and implementation, with no changes to the action class or JSP.

Maintainability

Each concern of the feature lives in exactly one place. The query logic is in the ORM XML files, the data-fetching logic is in the implementation class, the HTTP handling is in the action class, and the rendering is in the JSP. A developer debugging a display issue looks only at the JSP; one investigating a slow query looks only at the ORM XML. No layer mixes responsibilities with another.

Modularity

The feature is built as a vertical slice through the existing layered architecture without coupling any layer to the others. The JSP does not know how data is fetched. The action class does not know about JPA or SQL. The ORM queries do not know about the HTTP request. Changes in any one layer do not ripple into the others.

Feature 2 - Transforming Feeds

This feature introduces a post-processing pipeline that every blog entry passes through before being saved to the database. The pipeline automatically cleans, enriches, and annotates content without any manual intervention from the author or admin. It is implemented as an admin-side control and authors have no visibility into or control over the pipeline steps.

Design Pattern: Chain of Responsibility

The pipeline is implemented using the Chain of Responsibility design pattern. Each processing step is an independent unit that:

- Receives the WeblogEntry object
- Performs its own transformation on the entry
- Passes the entry to the next step in the chain via processNext(entry)

No step knows or depends on any other step. The chain is assembled in a single place (EntryProcessingPipeline.java). Each step only knows about the abstract interface it implements.

1.1 Class Structure

```
EntryProcessingStep      (interface)
|
AbstractEntryProcessingStep  (abstract base class)
|
|--- ReadingTimeEstimatorStep
|--- ProfanityFilterStep
|--- TextSummarizerStep
|--- AutoTagGeneratorStep
```

EntryProcessingPipeline (assembles and runs the chain)

1.2 Why Chain of Responsibility?

We require that steps can be added or removed without affecting the remaining process. Chain of Responsibility achieves this because:

- Each step only calls processNext(entry) to pass control forward
- Adding a new step requires adding one line to buildDefaultPipeline()
- Removing a step requires removing one line from buildDefaultPipeline()
- No other class needs to change in either case

2. Pipeline Steps

The pipeline executes steps in the following order:

Order	Step	Method	API Required
1	Reading Time Estimator	Word count calculation	No
2	Profanity Filter	Static word blacklist + regex	No
3	Text Summarizer	Extractive (default) or Groq LLM	Optional
4	Auto Tag Generator	Word frequency analysis (TF-style)	No

2.1 Step 1: Reading Time Estimator

File: ReadingTimeEstimatorStep.java

What it does:

- Counts the total word count of the blog entry
- Calculates estimated reading time based on average adult reading speed of 225 words per minute
- Prepends a [X min read | Y words] badge to the top of the entry body

Why it runs first: Reading time is calculated on the original full-length text before summarization, so the word count accurately reflects the actual post length.

Example output at the top of every post:

[3 min read | 712 words]

Artificial intelligence is rapidly transforming...

2.2 Step 2: Profanity Filter

File: ProfanityFilterStep.java

What it does:

- Scans the entry title, body, and summary for banned words
- Replaces each banned word with asterisks (****) using whole-word, case-insensitive regex matching
- Covers multiple categories of harmful content

Category	Examples
Violence	murder, kill, assault, torture, rape, abuse
Profanity	fuck, shit, bitch, bastard, asshole, damn
Spam indicators	spam, scam, phishing, clickbait
Drug references	cocaine, heroin, meth, crack

Adding or removing words only requires editing the BANNED_WORDS set in ProfanityFilterStep.java. No other class is affected.

Example:

Input: "This technology is complete bullshit"

Output: "This technology is complete *****"

2.3 Step 3: Text Summarizer

File: TextSummarizerStep.java

What it does:

- If the entry exceeds the configured max word count (default: 200 words), generates a summary
- Stores the summary in the entry's dedicated summary field, NOT the body
- The body text is not modified, it retains the full profanity-filtered content
- Roller uses the summary field for post previews on the blog homepage
- Word count is enforced with a hard trim before storing

Configuration in roller-custom.properties:

```
pipeline.summarizer.mode=extractive
pipeline.summarizer.maxwords=200
# pipeline.summarizer.mode=groq
# pipeline.summarizer.groq.apikey=API_KEY
```

Switching providers requires changing one line in roller-custom.properties. No code changes are required.

2.4 Step 4: Auto Tag Generator

File: AutoTagGeneratorStep.java

What it does:

Tokenizes the entry text into individual words

- Filters out common English stop words (the, and, for, etc.)
- Filters out short words (less than 4 characters)
- Counts word frequencies across the full text
- Extracts the top 5 most frequent meaningful words as tags
- Appends tags as plain text at the bottom of the entry body

Why it runs last: AutoTagGenerator runs after the summarizer so its output is not overwritten by any subsequent step.

Example output appended to body:

Tags: intelligence artificial healthcare learning systems

3. Integration Point

The pipeline is hooked into the existing saveWeblogEntry() method in JPAWeblogEntryManagerImpl.java. This is the only change made to existing code:

```
// Feature 2: Transforming Feeds Pipeline
if ((entry.getStatus() == WeblogEntry.PubStatus.PUBLISHED
    || entry.getStatus() == WeblogEntry.PubStatus.PENDING)
    && entry.getText() != null
    && entry.getText().trim().split("\\s+").length > 10) {
    EntryProcessingPipeline pipeline =
        EntryProcessingPipeline.buildDefaultPipeline();
    pipeline.process(entry);
    extractAndSetInlineTags(entry);
}
```

The pipeline only runs on PUBLISHED and PENDING entries drafts are not processed.

4. Files Added

All new files are placed in:

app/src/main/java/org/apache/roller/weblogger/business/pipeline/

File	Purpose
EntryProcessingStep.java	Interface defining the step contract
AbstractEntryProcessingStep.java	Base class handling chain wiring
ReadingTimeEstimatorStep.java	Step 1: reading time badge

ProfanityFilterStep.java	Step 2: profanity and harmful word filter
TextSummarizerStep.java	Step 3: auto summarization to summary field
AutoTagGeneratorStep.java	Step 4: keyword tag extraction
EntryProcessingPipeline.java	Assembles and executes the chain

Configuration file:

app/src/main/resources/org/apache/roller/weblogger/config/roller-custom.properties

5. How to Add or Remove a Step

5.1 Adding a New Step

- Create a new class extending AbstractEntryProcessingStep
- Implement the process(WeblogEntry entry) method with the logic
- Call processNext(entry) at the end of the method
- Add one line to buildDefaultPipeline() in EntryProcessingPipeline.java

No other file needs to change. This ensures that we are able to add/remove steps without it affecting the remaining process

5.2 Removing a Step

Remove the corresponding steps.add(...) line from buildDefaultPipeline() in EntryProcessingPipeline.java. No other file needs to change.

Bug Reports (Feature 3)

Design Pattern: Observer Pattern (Notification System)

Implementation: The notification system is built around the Observer pattern with:

- NotificationChannel (interface), the Observer
- NotificationService, the Subject/Publisher
- EmailNotificationChannel, concrete Observer implementation

Rationale: When a bug is submitted, resolved, or deleted, NotificationService iterates over all registered channels and calls sendNotification(). Adding Slack or MS Teams requires only implementing NotificationChannel and registering it, zero changes to existing code. This satisfies the Open/Closed Principle directly.

Design decisions:

- BugReport entity uses enums for Status and BugType, prevents invalid states at compile time
- BugReportManager interface decouples business logic from JPA implementation, swap persistence layer without touching UI
- JPABugReportManagerImpl handles all DB operations via JPA/ORM, no raw SQL, portable across databases
- isWeblogRequired() overridden to false in both actions, bug reporting is system-wide, not weblog-scoped

Quality attributes:

- *Extensibility*: New notification channels require no modification to existing channels
- *Maintainability*: Each channel is self-contained, independently testable
- *Modularity*: UI, business logic, persistence, and notification are fully separated layers

Trade-offs:

- Observer pattern adds indirection, slightly harder to trace execution flow
- Email delivery depends on third-party API (MailerSend), trial plan restricts recipients to verified addresses; mitigated by correct code design and documented in deployment notes, therefore switched to (Brevo)

Admin Dashboard -Feature 4

Overview

The Admin Dashboard is a site-wide monitoring feature implemented for Apache Roller administrators. It provides a centralized view of key platform metrics that were previously scattered across multiple manager interfaces. The dashboard supports two switchable view modes - Minimal and Full - and is accessible exclusively to global admin users. The implementation applies two Gang-of-Four design patterns: Facade (Structural) and Strategy (Behavioural), and strictly follows the Roller MVC layered architecture.

Files Created

Seven new files were created as part of this feature.

AdminDashboardData.java is a plain data-transfer object that carries all collected metrics between the service layer, controller, and view. It contains only fields with getters and setters - no business logic.

AdminDashboardFacade.java is the Facade class. It accepts three manager dependencies (UserManager, WeblogManager, WeblogEntryManager) and exposes a single public method, collectMetrics(), which coordinates all three subsystems and returns a populated AdminDashboardData object.

DashboardViewStrategy.java is the Strategy interface. It declares getMetricsToDisplay(), which returns the list of metric keys a view mode should render, and getViewLabel(), which returns a human-readable label.

MinimalViewStrategy.java is the concrete Strategy for Minimal View. It returns only two metric keys: totalUsers and topCategory. It contains zero data fetching logic.

FullViewStrategy.java is the concrete Strategy for Full View. It returns all nine metric keys. Like MinimalViewStrategy, it contains only a list declaration and a label string.

AdminDashboard.java is the Struts2 Action (controller). It constructs the Facade with the three managers, calls collectMetrics(), reads the viewMode request parameter, instantiates the appropriate Strategy, and stores the results for the JSP. It contains no metric fetching and no rendering logic.

AdminDashboard.jsp is the view. It renders each metric panel inside an <s:if test="visibleMetrics.contains('metricKey')"> check. It reads all values from dashboardData and never calls any manager or service directly.

Files Modified

Five existing files were modified.

struts.xml: The adminDashboard action was added inside the weblogger-admin package, after the bugDashboard action, registering the URL /roller-ui/admin/adminDashboard and mapping it to the AdminDashboard action class.

admin-menu.xml: A new menu-item for adminDashboard was added after the bugDashboard menu-item inside the admin menu block. It inherits the globalPerms="admin" restriction from the parent menu element automatically.

tiles.xml: A new .AdminDashboard tile definition was added after the .BugDashboard definition, extending .tiles-tabbedpage so the dashboard renders inside the standard Roller admin tabbed layout.

MainMenuSidebar.jsp: An Admin Dashboard link was added inside the existing userIsAdmin conditional block, after the Server Administration link, making it visible only to administrators on the Roller front page sidebar.

ApplicationResources.properties: The i18n key `tabbedmenu.admin.adminDashboard=Admin Dashboard` was added after the `bugDashboard` key to provide the display label for the admin navigation tab.

Metrics Implemented

Nine metrics are collected across the three manager subsystems. Total users and top performing categories are shown in both Minimal and Full view. The remaining seven - total weblogs, total entries, total comments, most starred blog, most starred entry, top three most active users, and most commented blog - are shown in Full view only.

Design Pattern Justification

Facade Pattern

Problem: The dashboard requires data from three separate manager subsystems. Without a Facade, the AdminDashboard action class would need to import all three managers, call each individually, handle their exceptions separately, and understand the internal structure of each subsystem. This creates tight coupling between the controller and three different interfaces. Any change to how a metric is fetched would require modifying the controller, blurring its responsibility from coordination into data retrieval.

Why Facade solves it: AdminDashboardFacade provides a single unified entry point. The controller calls only one method - `collectMetrics()` - and receives a fully populated data object. All complexity about which manager provides which metric, what exceptions can occur, and how to aggregate data across subsystems is hidden behind the Facade. The controller is completely decoupled from subsystem internals.

Alternative design: Without Facade, the `execute()` method in AdminDashboard.java would contain approximately 80 to 100 lines of manager calls, try-catch blocks, and aggregation logic directly. Every metric change would require opening and modifying the action class, violating the Single Responsibility Principle.

Key quality attribute improved - Maintainability: When a manager method changes, only AdminDashboardFacade needs updating. The controller and JSP are unaffected. A new metric requires changing only the Facade - zero changes to the controller or view.

Tradeoff: The indirection through the Facade increases the number of classes from one monolithic action to seven total files. New developers must understand the pattern to navigate the code. The Facade also risks becoming a large class as metrics grow, though this is mitigated by grouping metric fetching by subsystem with clear inline comments.

Strategy Pattern

Problem: The dashboard must support two display modes. The naive solution is to add a `viewMode` string and write conditional if-else logic in the JSP or controller to decide what to show. This has three problems. First, the definition of what each view shows is tangled with rendering or control logic rather than declared as an isolated, reusable object. Second, adding a third view mode requires modifying existing if-else blocks, violating the Open/Closed Principle. Third, and most critically, the assignment explicitly states that the definition of each view should not be described with the logic to fetch data, so that views can be flexibly modified independently. A conditional branch in the JSP or controller cannot satisfy this - it is not a definition, it is inline logic.

Why Strategy solves it: `DashboardViewStrategy` is an interface with one key method: `getMetricsToDisplay()`. `MinimalViewStrategy` returns a list of two keys. `FullViewStrategy` returns all nine keys. Neither class contains a single line of data fetching - they are purely declarative. The controller selects the strategy based on the request parameter and stores `visibleMetrics` for the JSP. The JSP wraps each panel in a visibility check against that list. This means the view definition (what to show) and data fetching (how to get it) are completely separated. The JSP does not know why a metric is visible - only whether it is.

Alternative design: A `viewMode` string checked with multiple conditionals in the JSP. Every new view requires modifying the JSP. The definition of what each view contains is not a reusable, testable object - it is scattered across conditional branches in the template.

Key quality attribute improved - Extensibility: Adding a new view mode requires only implementing `DashboardViewStrategy`. Zero changes are needed in the controller, the JSP, or any existing strategy class. This is the Open/Closed Principle in direct practice.

Tradeoff: For only two view modes, two extra classes may seem like over-engineering. Additionally, the metric keys used in Strategy classes and checked in the JSP are plain strings rather than a typed enum. A typo in a strategy or JSP silently hides a panel rather than producing a compile-time error. Introducing an enum would improve type safety at the cost of adding another file.

Overall Quality Attributes

Separation of Concerns is the primary quality attribute improved by this feature as a whole. The bold requirement from the specification is satisfied architecturally: Strategy classes contain only declarative lists, the Facade contains only data access coordination, and the JSP contains only rendering. No class performs more than one of these responsibilities.

Testability is improved because the Facade accepts its three manager dependencies via constructor, making it straightforward to unit test with mock managers. The Strategy classes are pure functions with no dependencies and are trivially unit testable. The tradeoff is that in the current implementation, the Facade is instantiated directly inside the `execute()` method rather than injected via Guice, meaning the action class itself is harder to isolate in unit tests. Injecting the Facade via the Guice module would resolve this in a future improvement.

Security is improved by enforcing admin-only access at three independent layers: the namespace security interceptor, the menu XML permission attribute, and the JSP conditional block. The tradeoff is minor redundancy, but for an admin feature, defence-in-depth is a deliberate and appropriate choice.

Performance relies on existing indexed JPA named queries for most metrics. The tradeoff is that the top active users metric retrieves all weblogs and permissions to aggregate entry counts per user, which could be expensive on very large installations. A dedicated JPQL aggregate query would be more efficient but was not added to avoid modifying existing manager implementations. This is a known candidate for future optimisation.

Feature 5A: Web Translation

Feature overview

The translation feature adds in-page language translation to Apache Roller weblogs, allowing blog readers to translate visible text content into their preferred language without reloading the page. It supports Sarvam AI (neural MT, India-focused) and Lingva (Google-backed, no API key needed) as two architecturally distinct translation backends, switchable via configuration.

Feature Type	Application-sided translation (not browser extension)
Translation Trigger	User clicks Translate button in floating toolbar
Text Replacement	DOM text nodes replaced in-place; all HTML structure preserved
Providers	Sarvam AI (neural MT) + Lingva Translate (Google-backed)
Languages Supported	10 Indic + European languages; all bidirectional
Provider Switching	Dropdown in UI; properties file; JVM system property
API Key Exposure	Zero - all API calls proxied through Java servlet (server-side only)
Restoration	One-click Original button restores all original text nodes

Code Changes

New Java Files Created

File	Type	Purpose
TranslationProvider.java	Interface (NEW)	Defines the contract all translation providers must implement: translate(), getProviderId(), getSupportedLanguages()

TranslationException.java	Exception (NEW)	Typed exception for translation failures - wraps API errors with meaningful messages
TranslationProviderFactory.java	Factory (NEW)	Reads translation.provider system property and instantiates the correct provider at runtime - zero code change to switch providers
SarvamTranslationProvider.java	Concrete Class (NEW)	Implements TranslationProvider using Sarvam AI REST translate endpoint - handles BCP-47 code mapping, chunking, JSON parsing
LingvaTranslationProvider.java	Concrete Class (NEW)	Implements TranslationProvider using Lingva (Google-backed) - a fundamentally different translation engine from Sarvam
TranslationServlet.java	Servlet (NEW)	HTTP REST endpoint at /roller-services/translation - receives POST requests from frontend, routes to provider, returns JSON

Modified Files

File	Type	Purpose
web.xml	Config (MODIFIED)	Added TranslationServlet definition and mapping to /roller-services/translation/* (public path, no auth required)
webblog.vm	Template (MODIFIED)	Added single script tag to load roller-translate.js - only change to existing template files
roller-custom.properties	Config (MODIFIED)	Added translation.sarvam.apikey and translation.lingva.instance - externalized all credentials

New Frontend File

File	Type	Purpose
roller-translate.js	JavaScript (NEW)	Floating translation toolbar UI; DOM text-node walker; deduplication; concurrent API calls; in-place text replacement; original restore

Quality Attributes

1. Extensibility

Adding a new translation provider requires zero changes to existing code. A developer creates a new class implementing `TranslationProvider`, adds one case to the factory switch, and the entire system - servlet, frontend, UI - automatically supports the new provider without touching any other file.

Evidence:

- `TranslationProvider` interface defines the exact contract: `translate`, `getProviderId`, `getSupportedLanguages`
- `TranslationProviderFactory` uses a switch on a config string - new case = new provider
- `TranslationServlet` has no knowledge of any specific provider; it only calls the interface
- Frontend dropdown can list any provider returned by the backend

2. Maintainability

Each provider is fully self-contained in its own class. A bug in Sarvam translation (e.g., API format change) is fixed only in `SarvamTranslationProvider.java` - no other file is affected. The servlet, the factory, and the frontend are all unmodified.

Evidence:

- `SarvamTranslationProvider`: 234 lines - entirely self-contained, zero dependencies on Lingva code
- `LingvaTranslationProvider`: 479 lines - entirely self-contained, zero dependencies on Sarvam code
- `TranslationException` provides typed error propagation - no stringly-typed error handling
- `roller-custom.properties` externalizes all API keys - no hardcoded credentials in source code

3. Modularity

The feature is organized into clearly separated layers - each with a single responsibility. The interface layer knows nothing about HTTP. The servlet layer knows nothing about Sarvam or Lingva. The factory layer knows nothing about translation logic. The frontend knows nothing about Java.

Evidence:

- `translation/` package: pure domain logic - no servlet, no HTTP imports
- `TranslationServlet`: pure HTTP handling - no Sarvam/Lingva imports, only uses the interface
- `roller-translate.js`: pure UI/DOM logic - communicates only via a well-defined JSON API
- `weblog.vm` change: exactly one line added - minimal coupling to the template layer

4. Changeability

The translation provider can be changed at four different levels without touching source code or recompiling. This is the definition of low-change-cost for a volatile configuration decision.

Evidence:

- Level 1 UI: User selects provider in dropdown on every page load

- Level 2 Per-request: POST body includes optional provider field to override
- Level 3 Runtime: JVM system property -Dtranslation.provider=lingva
- Level 4 Config: translation.provider=lingva in roller-custom.properties + server restart

Design Patterns

Three GoF design patterns are implemented in this feature.

Pattern 1: Factory Method Pattern [Creational]

GoF Intent	Defining an interface for creating object but let subclasses decide which class to be instantiated (Also Known As: Virtual Constructor)
Explanation	Factory Pattern: Defining an interface for creating objects but let subclasses decide which class to be instantiated. Two key objects: Factory (Creator) and Product
Implementation Mapping	TranslationProviderFactory (Creator) -> TranslationProvider interface (Product) -> SarvamTranslationProvider / LingvaTranslationProvider (ConcreteProduct)

Rationale

The translation system must support multiple interchangeable provider backends. Without a factory, every caller would need to know concrete class names and instantiate them directly - tight coupling that means adding a new provider requires modifying all call sites.

The Factory pattern centralizes provider creation in TranslationProviderFactory. The servlet calls TranslationProviderFactory.getProvider() and works exclusively against the TranslationProvider interface - it has no import of SarvamTranslationProvider or LingvaTranslationProvider. The concrete class returned depends entirely on the translation.provider property.

Implementation Mapping

Product Interface	TranslationProvider.java - defines translate(), getProviderId(), getSupportedLanguages()
ConcreteProduct A	SarvamTranslationProvider.java - neural MT via Sarvam REST API
ConcreteProduct B	LingvaTranslationProvider.java - Google-backed translation via Lingva
Creator (Factory)	TranslationProviderFactory.getProvider() - reads system property, returns correct instance
Client	TranslationServlet.java - uses only TranslationProvider interface, never concrete classes

Code Evidence

```
// TranslationProviderFactory.java
switch (providerId) {
    case "lingva": instance = new LingvaTranslationProvider(); break;
    case "sarvam": default: instance = new SarvamTranslationProvider(); break;
}
```

```
// TranslationServlet.java - never imports concrete providers
TranslationProvider provider = TranslationProviderFactory.getProvider();
String translated = provider.translate(text, sourceLang, targetLang);
```

Quality Attributes Improved

Quality Attribute	How Achieved	Evidence
Extensibility	New provider = one new class + one new switch case. Zero existing files need modification	Adding GeminiTranslationProvider requires touching only Factory.java
Maintainability	Provider-specific bugs are isolated. Servlet code never changes when a provider changes	Sarvam API format change -> edit only SarvamTranslationProvider.java
Changeability	Active provider can be changed via config without recompile	-Dtranslation.provider=lingva switches provider at JVM startup

Trade-Offs

- Factory pattern adds an indirection layer - a new developer must trace Factory -> Interface -> Concrete to understand which class is actually used at runtime
- The singleton caching in TranslationProviderFactory.instance requires TranslationProviderFactory.reset() if the provider needs to change mid-session - slightly complicates testing
- Adding a new provider still requires modifying the factory switch statement - a pure Open/Closed solution would use a registry, but that would add further complexity for limited gain given the small number of providers

Pattern 2: Strategy Pattern [Behavioral]

GoF Intent	Define a family of algorithms, encapsulate each one and ensure they are interchangeable. Strategy lets algorithm change depending on the client who is using it (Also Known As: Policy)
Explanation	Strategy Pattern: Define a family of algorithms, encapsulate each one and ensure they are interchangeable. Two key objects: Context and Strategy
Implementation Mapping	TranslationServlet (Context) -> TranslationProvider (Strategy interface) -> SarvamTranslationProvider / LingvaTranslationProvider (ConcreteStrategy)

Rationale

The two translation providers are not just different implementations of the same API - they represent fundamentally different translation algorithms. Sarvam AI uses a dedicated neural machine translation model specifically trained for Indic languages with Indic-specific language codes (hi-IN, te-IN) and domain-optimized models. Lingva uses Google translation infrastructure accessed through a proxy, with a completely different request/response schema and language code format.

The Strategy pattern lets the system treat both as interchangeable translation algorithms from the perspective of the servlet (Context), while each algorithm fully encapsulates its own data structures, API format, error handling, and language code mapping. Eliminates conditional statements: Encapsulates behavior separately [Good solution for long method smell].

Without Strategy, the servlet would contain a giant if/else block checking the provider name and calling different methods - exactly the long method smell the pattern is designed to eliminate.

Implementation Mapping

Strategy Interface	TranslationProvider.java - AlgorithmInterface: translate(text, source, target)
ConcreteStrategy A	SarvamTranslationProvider - encapsulates Sarvam-specific algorithm (Indic MT model, -IN language codes, chunking at 900 chars)
ConcreteStrategy B	LingvaTranslationProvider - encapsulates Lingva-specific algorithm (different endpoint, different code format, different response schema)
Context	TranslationServlet.doPost() - configured with a strategy at runtime; calls strategy.translate() without knowing the algorithm
Runtime Switch	providerHint in request body allows per-request strategy selection - Context changes its ConcreteStrategy on the fly

Code Evidence

```
// TranslationServlet.java - Context selects strategy based on request
if (providerHint != null && !providerHint.isEmpty()) {
    System.setProperty("translation.provider", providerHint);
}
```

```

    TranslationProviderFactory.reset();
    provider = TranslationProviderFactory.getProvider();
}
String translated = provider.translate(text, sourceLang, targetLang);

```

Quality Attributes Improved

Quality Attribute	How Achieved	Evidence
Modularity	Each algorithm's data structures and API quirks are fully encapsulated in its own class	Sarvam's -IN language codes, chunking, and JSON parsing are invisible to the servlet
Reusability	Both ConcreteStrategy classes can be used independently in any other context such as tests or batch tools	SarvamTranslationProvider can be unit tested with no servlet involved
Changeability	User selects strategy per-request via UI dropdown - runtime strategy switching without restart	Frontend sends provider: lingva to switch algorithm for that request only

Trade-Offs

Classes must be aware of different strategies - the frontend and servlet both need to know valid provider names (sarvam, lingva), creating a soft coupling on string identifiers

Per-request strategy override in TranslationServlet (temporarily setting system property, resetting factory) is not thread-safe under high concurrency - a cleaner design would pass the provider ID directly to the factory rather than using a system property

If the number of variations are less - Don't overcomplicate! - with only 2 providers currently, a simple if/else might suffice, but the pattern pays off as more providers are added

Pattern 3: Chain of Responsibility Pattern [Behavioral]

GoF Intent	Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until one handles it (Also Known As: CoR, Chain of Command)
Explanation	CoR Pattern: Requests may have to be passed along a chain. Senders and receivers need decoupling. Key objects: Handler, ConcreteHandler and Client
Implementation Mapping	The multi-layer request handling pipeline: Browser -> roller-translate.js -> TranslationServlet -> TranslationProviderFactory -> ConcreteProvider -> External API

Rationale

The translation request does not travel from the browser directly to the final translation API. Instead it passes through a chain of handlers, each of which has a specific responsibility and decides whether to process or forward the request:

- Handler 1 roller-translate.js: Collects text nodes, deduplicates, and decides whether to call the server or use cache. If cached, the chain stops here - no further handlers needed.
- Handler 2 TranslationServlet: Validates the request (missing text? missing targetLang? returns error, chain stops). If valid, forwards to the factory layer.
- Handler 3 TranslationProviderFactory: Determines which concrete provider handles the request based on configuration. Forwards to the selected provider.
- Handler 4 ConcreteProvider (Sarvam/Lingva): Handles the actual translation. If the text is too long, it splits into chunks and processes each sub-request internally before returning.
- Handler 5 External API (Sarvam / Lingva endpoint): The final handler performs the translation and returns the result up the chain.

Sometimes a single task may require multiple steps to process. Each step in the process may decide if it needs to be further processed or not. The cache check in the frontend is particularly illustrative if the result is cached, the chain terminates at Handler 1 and the external API is never called.

Implementation Mapping

ConcreteHandler 1	roller-translate.js - checks cache; if hit, applies translation and stops chain
ConcreteHandler 2	TranslationServlet - validates HTTP request; returns 400/404 errors to stop chain early
ConcreteHandler 3	TranslationProviderFactory - selects and forwards to appropriate concrete provider
ConcreteHandler 4	SarvamTranslationProvider / LingvaTranslationProvider - performs translation or throws
Client	Browser user - initiates the translation request by clicking Translate

Code Evidence - Chain Termination

```
// TranslationServlet.java - Handler 2 stops chain on invalid request
if (text == null || text.isEmpty()) {
    sendError(response, SC_BAD_REQUEST, "Missing 'text' field");
    return; // chain stops here - provider never called
}
```

```
// roller-translate.js - Handler 1 stops chain when cache exists
if (cacheOnly && cacheHits > 0) {
```

```

    isTranslated = true;
    return; // chain stops - API never called
}

```

Quality Attributes Improved

Quality Attribute	How Achieved	Evidence
Extensibility	A new processing step such as content filtering, rate limiting, or logging can be inserted anywhere in the chain without modifying other handlers	Add a LoggingHandler between Servlet and Factory with zero changes to other layers
Maintainability	Reduced coupling - the browser does not know about Sarvam, the servlet does not know about the browser cache, the provider does not know about HTTP	Object does not need to worry about which other object handles request
Performance	Early termination: cached requests never reach the Java servlet; invalid requests never reach the provider	Cache hit at Handler 1 = 0 API calls, 0 network latency to server

Trade-Offs

- Receipt is not guaranteed: If any handler in the chain fails silently such as a network timeout at Handler 4, the user may see partial translation with no clear error
- Debugging a chain of 5 handlers is harder than debugging a direct function call - a developer must trace through browser JS, servlet, factory, provider, and external API to diagnose failures
- The chain is not formalized through a Handler interface with explicit successor references as the GoF structure shows - the chain exists architecturally but is not enforced by a common base class, which reduces the pattern's formal guarantees

Translation Caching (Feature 5B)

Design Pattern: Cache-Aside Pattern + Strategy Pattern (inherited from 5A)

Implementation: Caching is implemented entirely client-side in roller-translate.js using localStorage:

- **Cache key:** rtc::{version}::{pageURL}::{targetLang}::{provider}

- **Cache value:** { ts: timestamp, sections: { djb2hash -> translatedText } }
- **User config key:** rtc::userconfig::{pageURL} stores last used language + provider

What constitutes a significant change: A text node is considered significantly changed if its **djb2 hash** differs from the stored hash. This operates at segment granularity:

- Navigation, sidebar, footer: hash never changes - permanently cached
- Article body, headings: hash changes on edit - only those segments retranslated
- Form inputs, code blocks: excluded via SKIP_TAGS, never translated

Partial invalidation logic: On every translate call, nodes are split into:

- **Cache hits:** applied instantly, zero API calls
- **Cache misses:** fetched from API, then merged back into cache

After fetching, new translations are merged with existing cache entries, unchanged entries are preserved, and changed entries are overwritten. Cache grows incrementally smarter.

Full-page vs partial retranslation tradeoff: Full-page retranslation is never triggered. Partial invalidation is always preferred because:

- M changed segments: M API calls ($M \leq N$ always)
- Neither Sarvam AI nor Lingva supports true batch requests, so per-segment calling is unavoidable regardless
- Therefore M calls always costs less than N calls in both latency and API credits
- Full-page would only be justified with a batch API offering sub-linear cost scaling, not applicable here

Auto page-load translation: On page load, if a valid cache entry exists for the user's last-used config, the page is automatically translated with zero API calls. Only if content has changed are misses fetched.

Cache eviction: Entries older than 7 days are evicted when localStorage approaches capacity, using a timestamp stored with each entry.

Quality attributes:

- *Cost efficiency:* Eliminates redundant Sarvam API calls, critical given free-tier credit limits
- *Performance:* Cache hits apply instantly (no network round-trip)
- *Extensibility:* Cache layer is completely independent of translation provider, works with any future provider added to 5A
- *Maintainability:* Pure JS, no server changes required, no new dependencies

Trade-offs:

- localStorage is browser-local, cache is not shared across devices or users. A shared server-side cache (Redis) would improve multi-user efficiency but adds infrastructure complexity
- djb2 hash has theoretical collision probability, acceptable for translation cache use case where a false cache hit would show a slightly wrong translation rather than causing data loss
- Cache is per-origin, works correctly in single-domain deployments like Roller

Feature 6A:

A new Discussion Overview tab was added to the Apache Roller weblog editor. This allows blog authors to select any of their published entries and instantly see a high-level snapshot of reader comment activity - all using lightweight, classical methods with zero AI or machine learning involvement.

How It Works:

The author logs into the weblog editor, clicks the Discussion Overview tab under the "Create & Edit" section, picks an entry from a dropdown, and clicks "Analyse Comments." The system then fetches all approved comments for that entry, runs them through three analysis indicators, and displays the results in a clean results table.

The Three Indicators

Activity Level looks at how lively the discussion has been over time. It counts total approved comments, then calculates how many comments were posted per day on average (measured from the first to the last comment). Based on this rate, it classifies the discussion as Low (fewer than 1 comment per day), Moderate (1 to 5 per day), or High (more than 5 per day). Response Types categorizes what kinds of comments readers are leaving. It uses regex pattern matching to detect questions (any comment containing a question mark), debate-style comments (words like "disagree," "wrong," "however," "incorrect"), and appreciative feedback (words like "thanks," "great," "helpful," "awesome"). Anything not matching these patterns falls into an "Other" bucket. The result is displayed as a breakdown such as "Questions: 3 | Feedback/Appreciation: 7 | Debate: 1 | Other: 2."

Top Keywords reveals the recurring topics readers are actually talking about. It takes all the comment text, tokenizes it by splitting on non-letter characters, removes around 50 common English stop-words and any tokens shorter than 3 characters, and then returns the 5 most frequently appearing words along with their counts - for example, "java (8) | spring (5) | blog (4)."

Files Created

Six new Java classes were created inside a new package called pulse under the business layer. DiscussionResult.java is a simple data-holding class (POJO) that stores a label and its computed value for one indicator. DiscussionIndicator.java is the strategy interface that all indicators implement, defining the two methods every indicator must provide: a label and a compute method. CommentActivityIndicator.java, CommentTypeIndicator.java, and KeywordFrequencyIndicator.java are the three concrete implementations, each handling one of the indicators described above. DiscussionAnalyzer.java is the orchestrator that holds a list of all indicators and runs them all together when analysis is requested.

On the UI side, DiscussionOverview.java was created as the Struts2 action class. It is responsible for loading the author's published entries, fetching approved comments for whichever entry the author selected, and calling the analyzer. DiscussionOverview.jsp was created as the actual page the author sees - it renders the entry-selector dropdown, the Analyse Comments button, the results table, and a legend explaining the indicators.

Files Modified and Why

struts.xml was modified to register the new discussionOverview action under the existing weblogger-authoring namespace, which is necessary for Struts2 to know how to route requests to the new action class.

tiles.xml was modified to add a tile definition for the Discussion Overview page. Apache Roller uses Apache Tiles for page layout composition, so without this entry the JSP cannot be rendered within the standard editor chrome.

editor-menu.xml was modified to add a new menu item pointing to the discussionOverview action inside the "Create & Edit" tab. This is what makes the new tab actually appear in the editor navigation for authors.

ApplicationResources.properties was modified to add 10 new internationalization keys. Apache Roller uses this properties file for all display text, so all labels, headings, and button text for the new page are defined here rather than hardcoded in the JSP.

Feature 6B - Conversation Breakdown

Feature Overview

Feature 6B extends the Apache Roller Community Pulse Dashboard with an AI-assisted Conversation Breakdown capability. For any selected blog entry, the feature provides authors with a structured, readable analysis of their comment section without requiring them to read every individual comment.

The system produces three structured outputs:

- A small set of major themes being discussed across comments
- Representative comment excerpts for each identified theme
- A short overall recap of the conversation

Authors can select between two distinct analysis methods via a dropdown - a lightweight keyword-based approach (instant, free, no API dependency) and an LLM-powered approach using the Groq API (richer semantic results, used only on demand).

Changes Made to the Existing Codebase

New Files Created

File Path	Description
business/pulse/ConversationBreakdownService.java	Strategy interface
business/pulse/ConversationBreakdown.java	Result POJO with inner CommentTheme class
business/pulse/LightweightBreakdownService.java	Lightweight keyword clustering strategy
business/pulse/LLMBreakdownService.java	Groq LLM strategy with fallback
business/pulse/ConversationBreakdownFactory.java	Factory for strategy selection and key management
ui/struts2/editor/ConversationBreakdownAction.java	Struts2 controller action
webapp/WEB-INF/jsp/editor/ConversationBreakdown.jsp	View: form + structured results

Modifications to Existing Files

File	Change Made
app/src/main/resources/struts.xml	Added <action name="conversationBreakdown"> block pointing to ConversationBreakdownAction
app/src/main/webapp/WEB-INF/tiles.xml	Added <definition name=".ConversationBreakdown"> tile extending .tiles-tabbedpage
app/src/main/resources/.../editor-menu.xml	Added <menu-item action="conversationBreakdown"> in the tabbedmenu.weblog section alongside discussionOverview
app/src/main/resources/ApplicationResources.properties	Added 14 new i18n keys for titles, labels, prompts, hints, and error messages under the conversationBreakdown.* namespace

The Two Analysis Methods

Method 1 - Lightweight (Keyword Clustering)

Pipeline

The lightweight method uses entirely classical, computationally inexpensive techniques:

1. Tokenisation & stop-word removal

Comment text is lowercased, split on non-alphabetic characters, and filtered against a hand-curated stop-word list. Words shorter than 3 characters are also discarded.

2. Small-set path (fewer than 5 comments)

TF-IDF is mathematically unreliable when the document count is very small (all IDF scores become equal). For fewer than 5 comments, each comment is treated as its own theme. The theme label is the most frequent meaningful word in that comment, falling back to the first 4 words if no keyword is found or the label would duplicate an existing theme.

3. Large-set path (5+ comments) - TF-IDF clustering

Each comment is assigned its highest-scoring TF-IDF keyword using a smoothed IDF formula: $IDF(t) = \log(1 + N / df(t))$. Comments sharing the same top keyword are grouped into a cluster (theme). Clusters are sorted by size and capped at 4 themes. Up to 2 representative comment excerpts are extracted per theme.

4. Recap generation

A recap sentence is assembled from simple statistics: total comment count, top theme name, and a comma-separated list of all theme labels. No model is involved.

Method 2 - LLM (Groq, llama-3.1-8b-instant)

Pipeline

5. Pre-processing (classical - no API cost)

Before any API call: HTML tags are stripped, comments over 300 characters are truncated, and the list is capped at 30 comments. This minimises token count and therefore cost and latency.

6. Single batch Groq API call

All pre-processed comments are sent in a single request. The prompt enforces a strict structured format (RECAP: / THEME N: / REP:) so the response is parseable with simple regex - no JSON library or second API call needed. Temperature is set to 0.3 for consistent, structured output.

7. Post-processing (classical - no API cost)

The raw response is parsed with regular expressions to extract THEME blocks, REP lines, and the RECAP paragraph. Accidental markdown symbols (asterisks, hashes) are stripped.

8. Fallback

If the API call fails (network error, quota exceeded, parse error), the service silently returns the lightweight result with a descriptive note in the method badge. The user always sees a result.

Groq API Integration

The Groq API is OpenAI-compatible (Chat Completions endpoint). No SDK dependency is added - the HTTP call uses only `java.net.HttpURLConnection` (already in the JDK):

POST <https://api.groq.com/openai/v1/chat/completions>

Authorization: Bearer <groq_api_key>

Content-Type: application/json

Design Patterns

Strategy Pattern

The Strategy pattern is the primary architectural pattern in this feature. The interface `ConversationBreakdownService` defines a uniform contract with two methods: `generate()`, which accepts a list of `WeblogEntryComment` objects and returns a `ConversationBreakdown` result, and `getMethodName()`, which returns a label identifying the strategy.

Two concrete strategies implement this interface:

- **LightweightBreakdownService** - classical keyword analysis, no external dependencies
- **LLMBreakdownService** - delegates to the Groq REST API

The Struts2 action (`ConversationBreakdownAction`) holds a reference only to the interface, obtained from the factory. The action calls `ConversationBreakdownFactory.getService(method)` to retrieve the appropriate strategy, then calls `service.generate(comments)` to produce the result, the same call regardless of which strategy was selected.

Rationale

The problem is a textbook Strategy scenario: the algorithm for generating a breakdown varies significantly between methods (keyword mathematics vs. HTTP API round-trip), but the input type (`List<WeblogEntryComment>`) and output type (`ConversationBreakdown`) are identical. Strategy separates what varies (the algorithm) from what stays constant (the request/response contract).

Quality Attributes

Extensibility: A third method (e.g., a local open-source model) requires one new class implementing the interface and one new case in the factory. Zero changes to the action, JSP, or existing strategies.

Maintainability: Each strategy is fully self-contained. Changes to the LLM prompt or the clustering algorithm are isolated to their respective files.

Testability: Each strategy can be unit-tested in complete isolation by calling `generate()` directly with mock comment lists.

Trade-offs

Indirection: The calling code never directly instantiates a service, which can make call-stack tracing slightly less obvious for a new developer.

Interface overhead: For exactly two strategies, the interface adds some boilerplate. This cost is justified because the assignment explicitly requires easy method switching.

Factory Method Pattern

ConversationBreakdownFactory provides a static factory method that maps the method string arriving from the UI form to the correct strategy instance. If the method equals llm (case-insensitive), it returns a new LLMBreakdownService initialised with the Groq API key; otherwise it returns a LightweightBreakdownService as the safe default.

The factory is also the single location that holds the Groq API key. The key is resolved in priority order: Java system property -> environment variable -> hardcoded fallback. This makes deployment-time key injection straightforward.

Rationale

Without the factory, the action would need to contain conditional logic and directly reference both concrete classes plus the API key. The factory separates the concern of object construction from the concern of orchestration.

Quality Attributes

Changeability: Changing which strategy is instantiated, or swapping the LLM provider entirely, requires modifying only ConversationBreakdownFactory.getService() - one place, not scattered across the codebase. The rest of the system is unaffected.

Extensibility: Adding a third breakdown method requires only one new implementing class and one new if-branch in the factory. Zero changes to the action, the JSP, or any existing strategy.

Reusability: The factory can be called from any future action or service that needs a breakdown strategy. Creation logic and key management do not need to be duplicated elsewhere.

Trade-offs

- For two strategies, a factory is lightweight overhead. Its value increases as more strategies are added.
- The static factory method pattern does not support dependency injection frameworks out of the box, but this is consistent with the existing Roller codebase which does not use DI for action classes.

CONTRIBUTIONS:

Akshitha:

- 2- Implementation of feed transformation
- Bonus

Rohitha:

- Implementation of AdminDashboard with Strategy and Facade Pattern(part 4)
- Discussion overview(part 6A)

Indu

- 1B Trending Blogs
- 6B Conversation Breakdown - Strategy and Factory Method patterns
- Bonus 2: Full Webpage Translation + A bug fixed in 5A & 5B

Vagdevi

- 1A Stars
- 5A Web Translation

Sherley

- Bug Reporting System (Feature 3) - Designed and implemented
- Translation Caching (Feature 5B) - Built client-side caching layer